# Drug Inventory & Supply Chain Tracking System

## 1. Project Overview

The Drug Inventory & Supply Chain Tracking System is a database-driven application designed to manage medicines, suppliers, stock levels, purchases, and sales in an efficient and structured way. The system helps in maintaining accurate inventory records, reducing manual errors, and improving decision-making in pharmacy or medical store operations.

This project focuses on strong database design, real-world implementation, and practical usability using MySQL and PHP.

## 2. Objectives

To design a structured database for medicine inventory management

To track stock levels of medicines in real time

To manage supplier information

To record purchase and sales transactions

To prevent stock shortages and manage expiry effectively

## 3. Technology Stack

Frontend:

HTML

CSS

Basic JavaScript

Backend:

PHP

Database:

MySQL (phpMyAdmin)

Environment:

XAMPP Server

## 4. System Architecture

The system follows a three-layer architecture:

1. Presentation Layer (Frontend): User interface created using HTML and CSS where users interact with the system.

2. Application Layer (Backend): PHP handles business logic, processes user input, and communicates with the database.

3. Data Layer (Database): MySQL database stores all system data including medicines, suppliers, purchases, and sales.

## 5. Database Design

The database is the core component of this system.

### 5.1 Tables Description

**Suppliers**

Stores supplier details.

Fields:

supplier_id (Primary Key)

supplier_name

contact

address

**Medicines**

Stores medicine information.

Fields:

medicine_id (Primary Key)

medicine_name

category

batch_no

expiry_date

purchase_price

selling_price

quantity

supplier_id (Foreign Key)

created_at

**Purchases**

Tracks all purchase transactions.

Fields:

purchase_id (Primary Key)

medicine_id (Foreign Key)

quantity

purchase_date

supplier_id (Foreign Key)

**Sales**

Tracks all sales transactions.

Fields:

sale_id (Primary Key)

medicine_id (Foreign Key)

quantity

sale_date

## 6. Entity Relationship Logic

One supplier can supply multiple medicines

One medicine can have multiple purchase records

One medicine can have multiple sales records

**Relationships:**

suppliers → medicines (1:N)

medicines → purchases (1:N)\

medicines → sales (1:N)

## 7. Functional Modules

**7.1 Supplier Management**

Add supplier

View supplier list

Store supplier contact details

**7.2 Medicine Management**

Add new medicines

Store batch number and expiry date

Maintain purchase and selling price

Track quantity in stock

**7.3 Purchase Module**

Record new purchases

Increase stock quantity automatically

Link supplier and medicine

**7.4 Sales Module**

Record sales transactions

Reduce stock quantity automatically

**7.5 Inventory Monitoring**

Track available stock

Identify low stock medicines

Monitor expiry dates

## 8. Workflow of the System

1. Admin adds supplier details

2. Admin adds medicines linked to suppliers

3. When stock is purchased:

Entry is made in purchases table

Quantity increases in medicines table

4. When a sale occurs:

Entry is made in sales table

Quantity decreases in medicines table

5. System continuously reflects updated stock

## 9. Key Features

Structured relational database

Real-time stock tracking

Supplier linkage

Purchase and sales history

Expiry date tracking

Simple and user-friendly interface

## 10. Advantages

Reduces manual errors

Improves inventory accuracy

Easy to maintain and scale

Helps in decision-making

Suitable for small to medium pharmacies

## 11. Limitations

No authentication system (basic version)

No barcode integration

Limited UI design

Not cloud-based

## 12. Future Enhancements

User login and role management

Barcode scanning system

Automated expiry alerts

Low stock notifications

Dashboard with analytics

Integration with billing system

## 13. Conclusion

This project demonstrates a practical implementation of database management and web development concepts. It highlights the importance of structured data handling, real-world problem solving, and efficient system design.

The system can be further expanded into a full-scale pharmacy management solution with advanced features and improved user interface.

## 14. Website Structure and Webpages Documentation

This section explains the complete structure of the website, including all pages, their purpose, and internal working logic.

### 14.1 Folder Structure

/project-folder |— config/ | └— db.php |— suppliers/ | ├— add_supplier.php | ├— view_suppliers.php |— medicines/ | ├— add_medicine.php | ├— view_medicines.php |— purchases/ | ├— add_purchase.php |— sales/ | ├— add_sale.php |— index.php |— navbar.php

### 14.2 Common File

**db.php (Database Connection)**

Purpose:

Establish connection between PHP and MySQL database

Working:

Uses mysqli_connect()

Stores connection in a variable

Included in all pages using include()

Example Logic:

Connect to localhost

Select database

If connection fails, show error

**14.3 Homepage**

index.php

Purpose:

Acts as dashboard/homepage

Features:

Navigation to all modules

Overview of system

Working:

Loads navbar

Displays links/buttons for:

Suppliers

Medicines

Purchases

Sales

**14.4 Supplier Module**

add_supplier.php

Purpose:

Add new supplier to database

Form Fields:

Supplier Name

Contact

Address

Working:

1. User fills form

2. On submit:

Data is collected using $_POST

SQL INSERT query executed

3. Supplier stored in suppliers table

Core Function:

INSERT INTO suppliers

view_suppliers.php

Purpose:

Display all suppliers

Working:

1. Fetch data using SELECT query

2. Loop through result using while loop

3. Display in HTML table

Core Function:

SELECT * FROM suppliers

**14.5 Medicine Module**

add_medicine.php

Purpose:

Add new medicine

Form Fields:

Medicine Name

Category

Batch No

Expiry Date

Purchase Price

Selling Price

Quantity

Supplier (dropdown)

Working:

1. Fetch suppliers for dropdown

2. User fills form

3. On submit:

Data captured via $_POST

INSERT query executed

4. Medicine stored in medicines table

Core Functions:

SELECT suppliers (for dropdown)

INSERT INTO medicines

view_medicines.php

Purpose:

View all medicines

Working:

1. Fetch medicines using SELECT

2. Join with suppliers table (optional improvement)

3. Display in table format

Core Function:

SELECT * FROM medicines

**14.6 Purchase Module**

add_purchase.php

Purpose:

Record purchase of medicines

Form Fields:

Medicine

Quantity

Supplier

Purchase Date

Working:

1. User selects medicine and enters quantity

2. On submit:

Insert record into purchases table

Update stock in medicines table

Core Logic:

INSERT INTO purchases

UPDATE medicines SET quantity = quantity + new_quantity

**14.7 Sales Module**

add_sale.php

Purpose:

Record sale of medicines

Form Fields:

Medicine

Quantity

Sale Dat

Working:

1. User selects medicine

2. On submit:

Insert record into sales table

Reduce stock in medicines table

Core Logic:

INSERT INTO sales

UPDATE medicines SET quantity = quantity - sold_quantity

**14.8 Navigation System**

navbar.php

Purpose:

Common navigation bar for all pages

Features:

Links to all modules

Working:

Included using include('navbar.php')

Ensures consistent UI across pages

**14.9 Data Flow Between Pages**

1. User interacts with forms
2. Data sent using POST method
3. PHP processes request
4. SQL queries executed
5. Database updated
6. Updated data shown on view pages

**14.10 Key Functions Used**

Database Functions

mysqli_connect()

mysqli_query()

mysqli_fetch_assoc()

PHP Superglobals

$_POST (form data)

$_GET (optional navigation)

SQL Operations

INSERT

SELECT

UPDATE

**15. GitHub README Version**

Drug Inventory & Supply Chain Tracking System

A simple, database-driven web application to manage medicines, suppliers, stock levels, purchases, and sales. Built using PHP and MySQL with a focus on clean database design and practical workflows.

🚀 Features

Add and manage suppliers

Add and manage medicines (batch, expiry, pricing)

Record purchases (auto-increase stock)

Record sales (auto-decrease stock)

View inventory in real-time

Structured relational database design

🧱 Tech Stack

Frontend: HTML, CSS, JavaScript

Backend: PHP

Database: MySQL (phpMyAdmin)

Server: XAMPP

📁 Project Structure

```
/project-folder
|— config/
|   └— db.php
|— suppliers/
|   ├— add_supplier.php
|   ├— view_suppliers.php
|— medicines/
|   ├— add_medicine.php
|   ├— view_medicines.php
|— purchases/
|   ├— add_purchase.php
|— sales/
|   ├— add_sale.php
|— index.php
|— navbar.php
```

⚙️ Setup Instructions

1. Install XAMPP
2. Start Apache and MySQL
3. Open phpMyAdmin
4. Create a database named:

drug_inventory

5. Import the SQL schema (tables: suppliers, medicines, purchases, sales)
6. Place the project folder inside:

C:/xampp/htdocs/

7. Open browser and run:

http://localhost/project-folder/

🗄️ Database Overview

Tables used:

suppliers

medicines

purchases

sales

Key Relationships:

One supplier → many medicines

One medicine → many purchases

One medicine → many sales

🔄 Core Workflow

1. Add Supplier
2. Add Medicine
3. Record Purchase → stock increases
4. Record Sale → stock decreases
5. View updated inventory anytime

🧠 Key Logic

Stock is maintained in the medicines table

Purchase adds quantity

Sale subtracts quantity

Foreign keys ensure relational integrity

📌 Future Improvements

Authentication system

Low stock alerts

Expiry notifications

Dashboard with analytics

Barcode integration

## 👨‍💻 Author

Gulshan (B.Tech IT)

## 📜 License

This project is for educational purposes.

## 16. Keywords

Drug Inventory, Supply Chain, Database Management, MySQL, PHP, Stock Management, Pharmacy System